Tugas Pertemuan 10.2 – Trigger

Mata Kuliah: Sistem Basis Data (2026)

Prodi: Sarjana Terapan-Teknik Informatika POLBAN

Kelas: 2B

NIM: 251524048

Nama Lengkap: Ghaisan Khoirul Badruzaman

Prasyarat

1. Menggunakan alat bantu atau perangkat lunak RDBMS PostgreSQL
2. Menggunakan skema yang diberikan untuk studi kasus pemesanan produk.

Instruksi

1. Total seluruh soal yang bisa dikerjakan sebanyak **3 soal**.
2. **Disarankan mengerjakan soal secara terturut**!
3. Sertakan jawaban dalam bentuk syntax PL/SQL!
4. Sertakan **screenshot halaman penuh yang menyertakan taskbar dengan waktu** (Print Screen) untuk menunjukkan syntax dan hasil eksekusi query pada RDBMS PostgreSQL!
5. Ekspor file lembar jawaban ini dalam format PDF dengan nama file:
   T10.2_<Kelas>_<3 NIM Terakhir>.pdf
   contoh sebagai berikut:
   T10.2_2A_010.pdf
6. Kumpulkan melalui Assignment pada Ms. Teams

Referensi

1. https://www.postgresql.org/docs/current/sql-createtrigger.html
2. https://neon.com/postgresql/postgresql-triggers

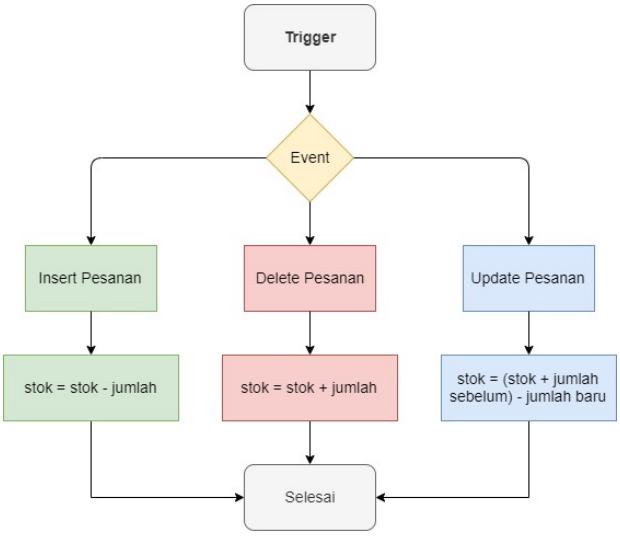
Jika ada hal yang ingin ditanyakan, silahkan sampaikan melalui WhatsApp Group/Ms. Teams.

Selamat mengerjakan!

Soal No. 1

Soal

Business Process #1



a. Buatlah terlebih dahulu tabel produk dan pesanan, disertai dengan data untuk masing-masing table, sesuai yang telah disediakan:

```
-- Membuat Tabel Produk
CREATE TABLE produk (
    id_produk NUMERIC NOT NULL,
    produk VARCHAR(20) NOT NULL,
    stok NUMERIC,
    satuan VARCHAR(10),
    CONSTRAINT produk_pk PRIMARY KEY (id_produk)
);


-- Membuat Tabel Pesanan
CREATE TABLE pesanan (
    id_pesanan NUMERIC NOT NULL,
    id_produk NUMERIC NOT NULL,
    jumlah NUMERIC NOT NULL,
    tgl_pesanan DATE,
    CONSTRAINT pesanan_pk PRIMARY KEY (id_pesanan)
);


-- Mengisi Data Produk
INSERT INTO produk VALUES (1, 'Choki-Choki', 20, 'pcs');
INSERT INTO produk VALUES (2, 'Potato Stick', 15, 'pcs');
INSERT INTO produk VALUES (3, 'Silver Queen', 25, 'pcs');
INSERT INTO produk VALUES (4, 'Pocky', 5, 'box');
```

b. Buatlah function yang akan digunakan oleh Trigger sebagai berikut:

```
-- Membuat Function untuk Trigger - INSERT, UPDATE, DELETE
CREATE OR REPLACE FUNCTION fn_update_stok()
  RETURNS TRIGGER
  LANGUAGE PLPGSQL
  AS
$$
BEGIN
IF TG_OP = 'INSERT' THEN
UPDATE produk SET stok = (stok - NEW.jumlah) WHERE id_produk = NEW.id_produk;
ELSIF TG_OP = 'UPDATE' THEN
UPDATE produk SET stok = ((stok + OLD.jumlah) - NEW.jumlah) WHERE id_produk =
    OLD.id_produk;
ELSIF TG_OP = 'DELETE' THEN
UPDATE produk SET stok = (stok + OLD.jumlah) WHERE id_produk = OLD.id_produk;
END IF;


RETURN NULL;
END;
$$
```

c. Buatlah Trigger yang akan memanggil function yang telah dibuat sebagai berikut:

```
-- Membuat Trigger yang memanggil function fn_update_stok()
CREATE OR REPLACE TRIGGER trg_update_stok
  AFTER INSERT OR UPDATE OR DELETE on pesanan
  FOR EACH ROW
  EXECUTE FUNCTION fn_update_stok();
```

d. Buatlah contoh query untuk menguji operasi INSERT!
e. Buatlah contoh query untuk menguji operasi UPDATE!
f. Buatlah contoh query untuk menguji operasi DELETE!

Jawaban (Penjelasan dan Syntax)

Persiapan: membuat database pemesanan lalu masuk ke database tersebut

Masuk ke psql sebagai user postgres, mematikan pager, lalu membuat database baru bernama pemesanan khusus untuk studi kasus ini dan pindah ke database tersebut.

```
sudo -u postgres psql

\pset pager off

CREATE DATABASE pemesanan;

\c pemesanan
```

Soal 1: membuat tabel produk dan pesanan beserta data produk

Tabel produk dan pesanan dibuat persis sesuai syntax dari soal, lalu tabel produk diisi 4 data awal: Choki-Choki 20 pcs, Potato Stick 15 pcs, Silver Queen 25 pcs, dan Pocky 5 box.

Tabel pesanan tidak punya foreign key ke produk karena syntax dari soal dipakai apa adanya.

```
-- Membuat Tabel Produk
CREATE TABLE produk (
    id_produk NUMERIC NOT NULL,
    produk VARCHAR(20) NOT NULL,
    stok NUMERIC,
    satuan VARCHAR(10),
    CONSTRAINT produk_pk PRIMARY KEY (id_produk)
);

-- Membuat Tabel Pesanan
CREATE TABLE pesanan (
    id_pesanan NUMERIC NOT NULL,
    id_produk NUMERIC NOT NULL,
    jumlah NUMERIC NOT NULL,
    tgl_pesanan DATE,
    CONSTRAINT pesanan_pk PRIMARY KEY (id_pesanan)
);

INSERT INTO produk VALUES (1, 'Choki-Choki', 20, 'pcs');
```

```
INSERT INTO produk VALUES (2, 'Potato Stick', 15, 'pcs');

INSERT INTO produk VALUES (3, 'Silver Queen', 25, 'pcs');

INSERT INTO produk VALUES (4, 'Pocky', 5, 'box');

SELECT * FROM produk ORDER BY id_produk;
```

Soal 1: membuat function fn_update_stok() dan trigger trg_update_stok

fn_update_stok adalah function khusus trigger: tidak punya parameter dan RETURNS TRIGGER. Di dalamnya variabel TG_OP berisi jenis operasi yang memicu trigger ('INSERT', 'UPDATE', atau 'DELETE'), NEW berisi baris baru (ada pada INSERT dan UPDATE), dan OLD berisi baris lama (ada pada UPDATE dan DELETE).

Rumusnya sesuai flowchart Business Process #1: INSERT mengurangi stok sebanyak NEW.jumlah, DELETE mengembalikan stok sebanyak OLD.jumlah, dan UPDATE mengembalikan jumlah lama dulu lalu mengurangi jumlah baru.

Trigger trg_update_stok dipasang AFTER INSERT OR UPDATE OR DELETE pada tabel pesanan dengan FOR EACH ROW, jadi function dijalankan sekali untuk setiap baris pesanan yang berubah. Karena trigger-nya AFTER, nilai RETURN diabaikan dan RETURN NULL sudah cukup. \d pesanan menunjukkan trigger sudah terpasang.

```
CREATE OR REPLACE FUNCTION fn_update_stok()
  RETURNS TRIGGER
  LANGUAGE PLPGSQL
  AS
$$
BEGIN
IF TG_OP = 'INSERT' THEN
UPDATE produk SET stok = (stok - NEW.jumlah) WHERE id_produk = NEW.id_produk;
ELSIF TG_OP = 'UPDATE' THEN
UPDATE produk SET stok = ((stok + OLD.jumlah) - NEW.jumlah) WHERE id_produk =
    OLD.id_produk;
ELSIF TG_OP = 'DELETE' THEN
UPDATE produk SET stok = (stok + OLD.jumlah) WHERE id_produk = OLD.id_produk;
END IF;

RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_update_stok
  AFTER INSERT OR UPDATE OR DELETE on pesanan
  FOR EACH ROW
  EXECUTE FUNCTION fn_update_stok();

\d pesanan
```

Soal 1: pengujian operasi INSERT

Pesanan 1 (Choki-Choki 5 pcs) dan pesanan 2 (Pocky 2 box) dimasukkan. Stok Choki-Choki berkurang dari 20 menjadi 15 dan Pocky dari 5 menjadi 3, padahal yang di-INSERT hanya tabel pesanan. Perubahan stok dilakukan otomatis oleh trigger.

```
INSERT INTO pesanan VALUES (1, 1, 5, '2026-11-11');

INSERT INTO pesanan VALUES (2, 4, 2, '2026-11-11');

SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;
```

Soal 1: pengujian operasi UPDATE

Jumlah pesanan 1 diubah dari 5 menjadi 8. Trigger menghitung (stok + OLD.jumlah) - NEW.jumlah = (15 + 5) - 8 = 12, jadi stok Choki-Choki menjadi 12. Produk lain tidak berubah.

```
UPDATE pesanan SET jumlah = 8 WHERE id_pesanan = 1;
```

```
SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;
```

Soal 1: pengujian operasi DELETE

Pesanan 2 (Pocky 2 box) dihapus. Trigger mengembalikan stok sebanyak OLD.jumlah, jadi stok Pocky kembali dari 3 menjadi 5.

```
DELETE FROM pesanan WHERE id_pesanan = 2;

SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;
```

Soal 1 (masalah yang muncul): stok bisa bernilai minus

Ini masalah di Business Process #1: pesanan 3 memesan Pocky 10 box padahal stoknya cuma 5. INSERT tetap diterima dan stok Pocky menjadi -5, karena function belum memeriksa ketersediaan stok.

Pesanan 3 lalu dihapus supaya stok Pocky kembali ke 5 sebelum solusi Business Process #2 diterapkan.

```
INSERT INTO pesanan VALUES (3, 4, 10, '2026-11-11');

SELECT * FROM pesanan ORDER BY id_pesanan;

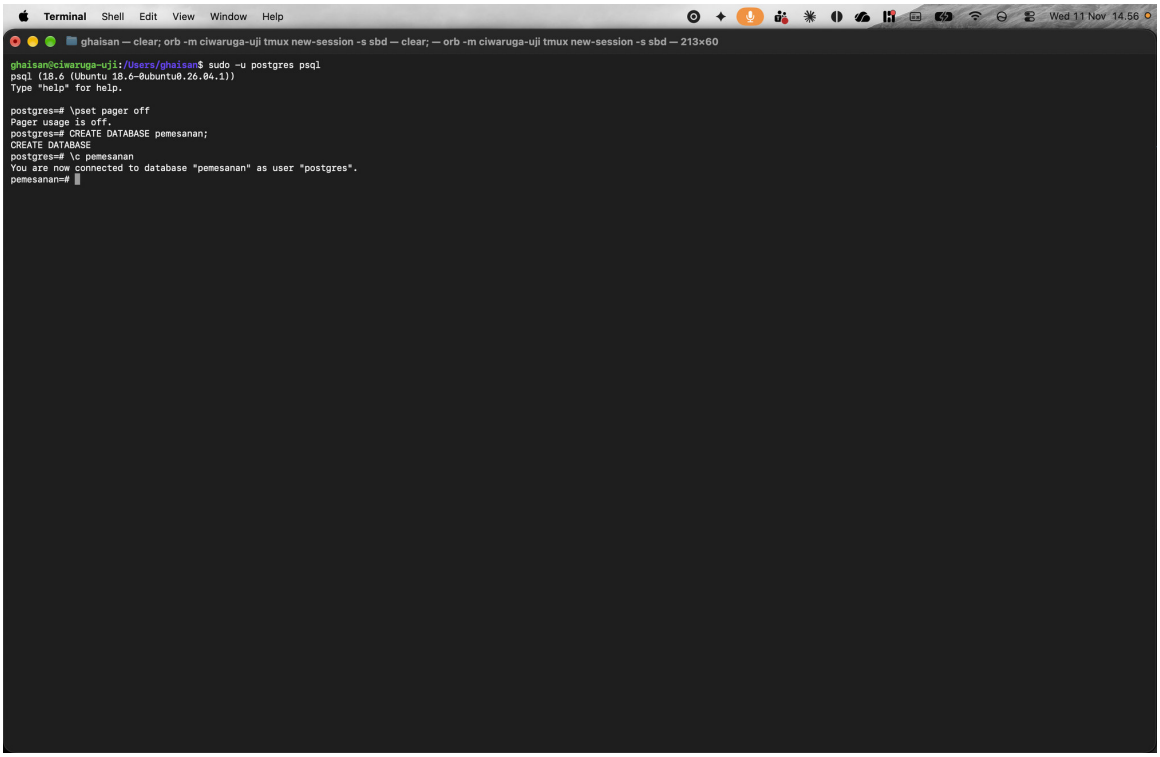
SELECT * FROM produk ORDER BY id_produk;

DELETE FROM pesanan WHERE id_pesanan = 3;

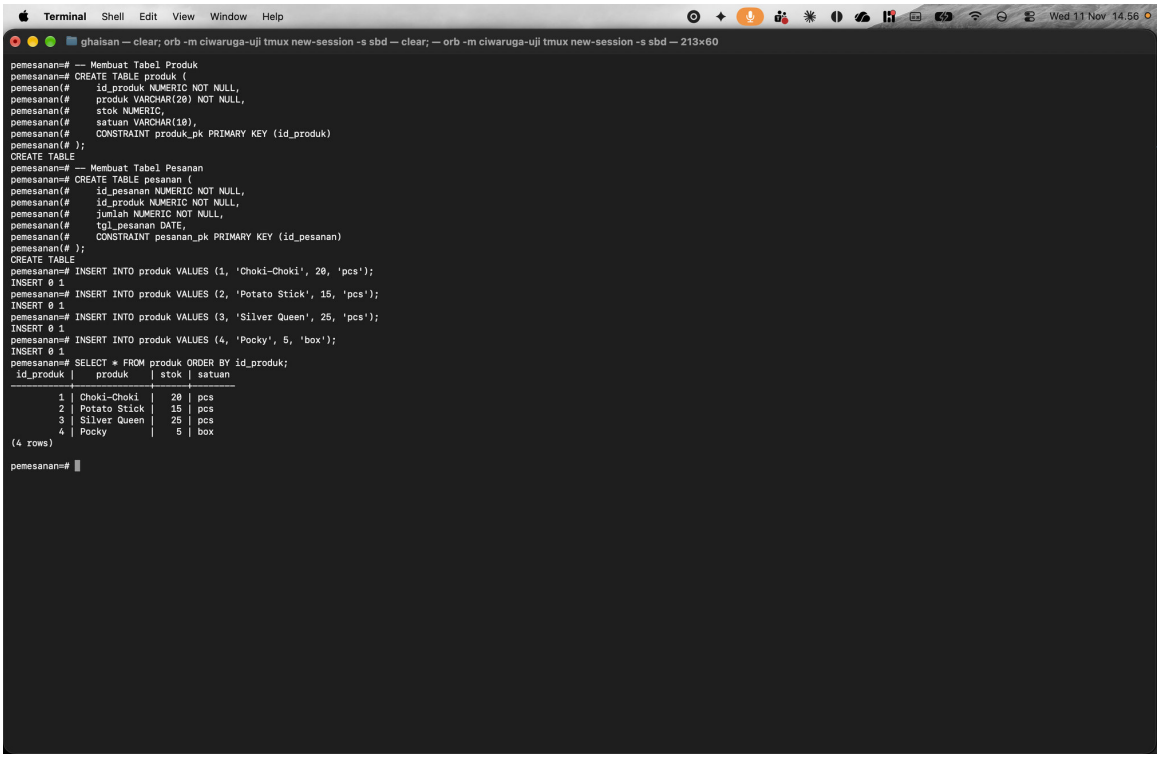
SELECT * FROM produk WHERE id_produk = 4;
```

Hasil Eksekusi (Screenshot)

Persiapan: membuat database pemesanan lalu masuk ke database tersebut



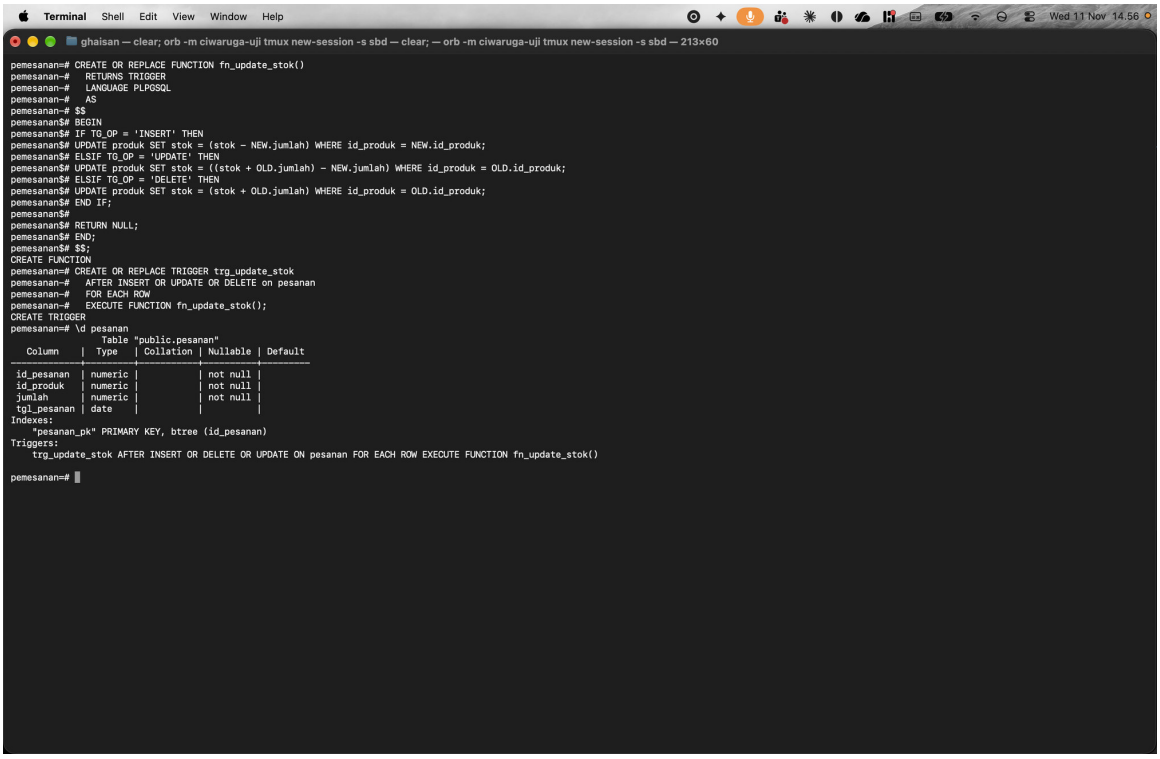
Soal 1: membuat tabel produk dan pesanan beserta data produk



```
pemesanan=# -- Membuat Tabel Produk
pemesanan=# CREATE TABLE produk (
pemesanan(#     id_produk NUMERIC NOT NULL,
pemesanan(#     produk VARCHAR(20) NOT NULL,
pemesanan(#     stok NUMERIC,
pemesanan(#     satuan VARCHAR(10),
pemesanan(#     CONSTRAINT produk_pk PRIMARY KEY (id_produk)
pemesanan(# );
CREATE TABLE
pemesanan=# -- Membuat Tabel Pesanan
pemesanan=# CREATE TABLE pesanan (
pemesanan(#     id_pesanan NUMERIC NOT NULL,
pemesanan(#     id_produk NUMERIC NOT NULL,
pemesanan(#     jumlah NUMERIC NOT NULL,
pemesanan(#     tgl_pesanan DATE,
pemesanan(#     CONSTRAINT pesanan_pk PRIMARY KEY (id_pesanan)
pemesanan(# );
CREATE TABLE
pemesanan=# INSERT INTO produk VALUES (1, 'Choki-Choki', 20, 'pcs');
INSERT 0 1
pemesanan=# INSERT INTO produk VALUES (2, 'Potato Stick', 15, 'pcs');
INSERT 0 1
pemesanan=# INSERT INTO produk VALUES (3, 'Silver Queen', 25, 'pcs');
INSERT 0 1
pemesanan=# INSERT INTO produk VALUES (4, 'Pocky', 5, 'box');
INSERT 0 1
pemesanan=# SELECT * FROM produk ORDER BY id_produk;
 id_produk |    produk    | stok | satuan
-----------+--------------+------+--------
         1 | Choki-Choki  |   20 | pcs
         2 | Potato Stick |   15 | pcs
         3 | Silver Queen |   25 | pcs
         4 | Pocky        |    5 | box
(4 rows)

pemesanan=#
```

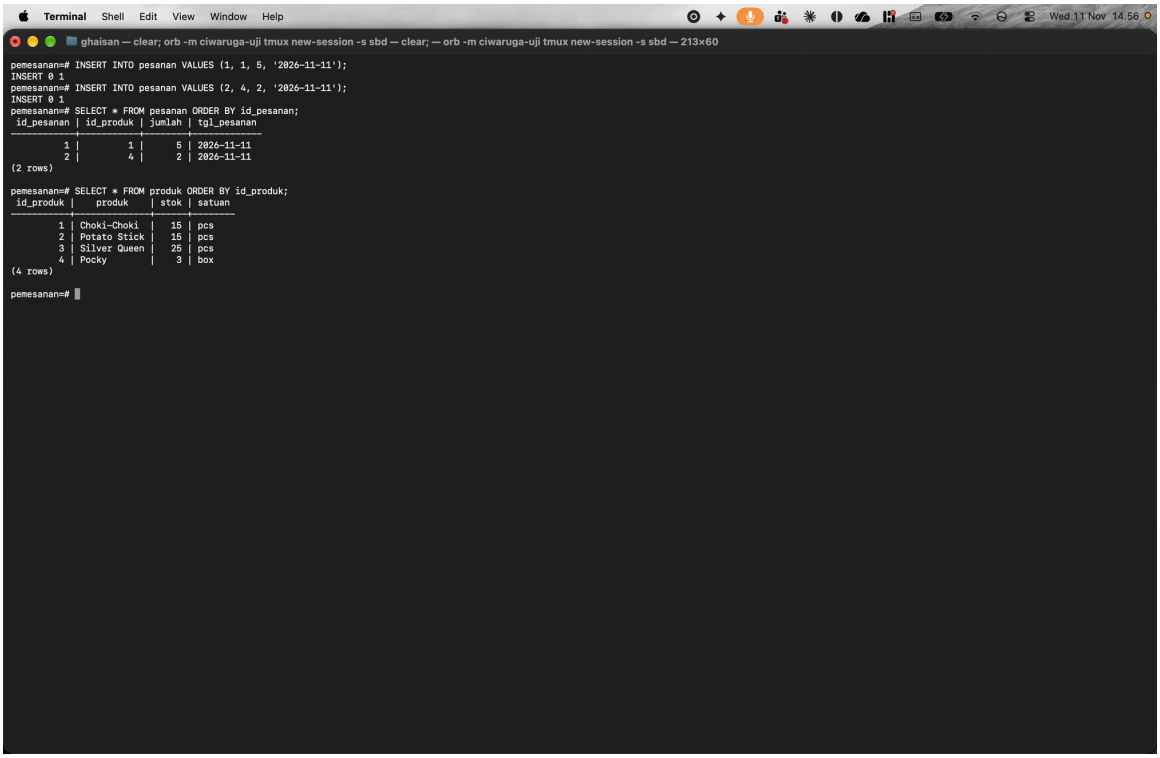
Soal 1: membuat function fn_update_stok() dan trigger trg_update_stok



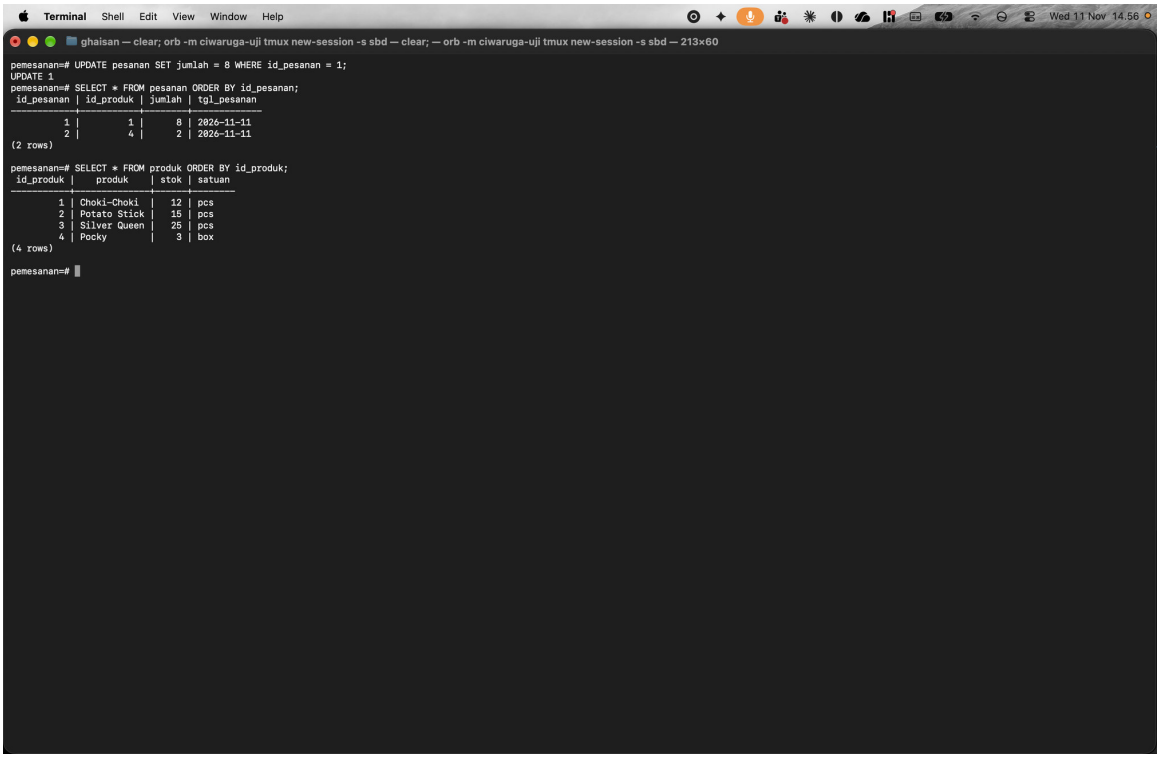
```
pemesanan=# CREATE OR REPLACE FUNCTION fn_update_stok()
pemesanan-#   RETURNS TRIGGER
pemesanan-#   LANGUAGE PLPGSQL
pemesanan-#   AS
pemesanan-# $$
pemesanan$# BEGIN
pemesanan$# IF TG_OP = 'INSERT' THEN
pemesanan$# UPDATE produk SET stok = (stok - NEW.jumlah) WHERE id_produk = NEW.id_produk;
pemesanan$# ELSIF TG_OP = 'UPDATE' THEN
pemesanan$# UPDATE produk SET stok = ((stok + OLD.jumlah) - NEW.jumlah) WHERE id_produk = OLD.id_produk;
pemesanan$# ELSIF TG_OP = 'DELETE' THEN
pemesanan$# UPDATE produk SET stok = (stok + OLD.jumlah) WHERE id_produk = OLD.id_produk;
pemesanan$# END IF;
pemesanan$#
pemesanan$# RETURN NULL;
pemesanan$# END;
pemesanan$# $$;
CREATE FUNCTION
pemesanan=# CREATE OR REPLACE TRIGGER trg_update_stok
pemesanan-#   AFTER INSERT OR UPDATE OR DELETE on pesanan
pemesanan-#   FOR EACH ROW
pemesanan-#   EXECUTE FUNCTION fn_update_stok();
CREATE TRIGGER
pemesanan=# \d pesanan
                Table "public.pesanan"
   Column    |  Type   | Collation | Nullable | Default
-------------+---------+-----------+----------+---------
 id_pesanan  | numeric |           | not null |
 id_produk   | numeric |           | not null |
 jumlah      | numeric |           | not null |
 tgl_pesanan | date    |           |          |
Indexes:
    "pesanan_pk" PRIMARY KEY, btree (id_pesanan)
Triggers:
    trg_update_stok AFTER INSERT OR DELETE OR UPDATE ON pesanan FOR EACH ROW EXECUTE FUNCTION fn_update_stok()

pemesanan=#
```

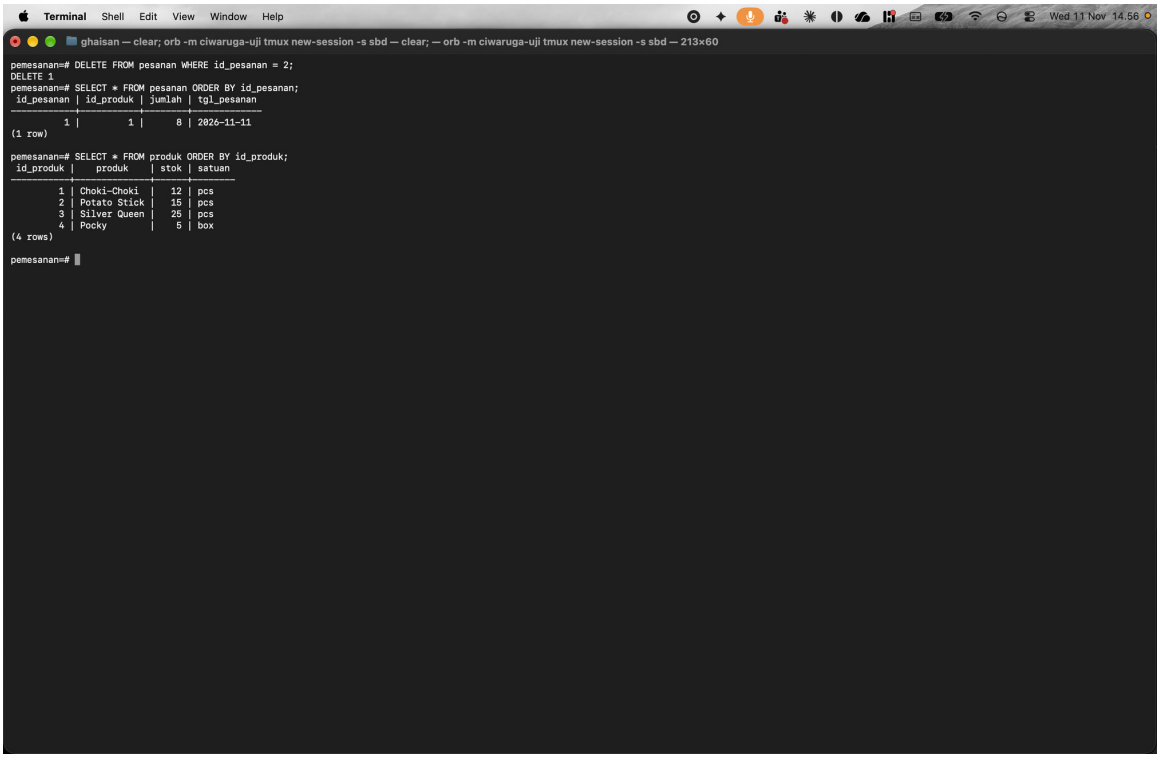
Soal 1: pengujian operasi INSERT



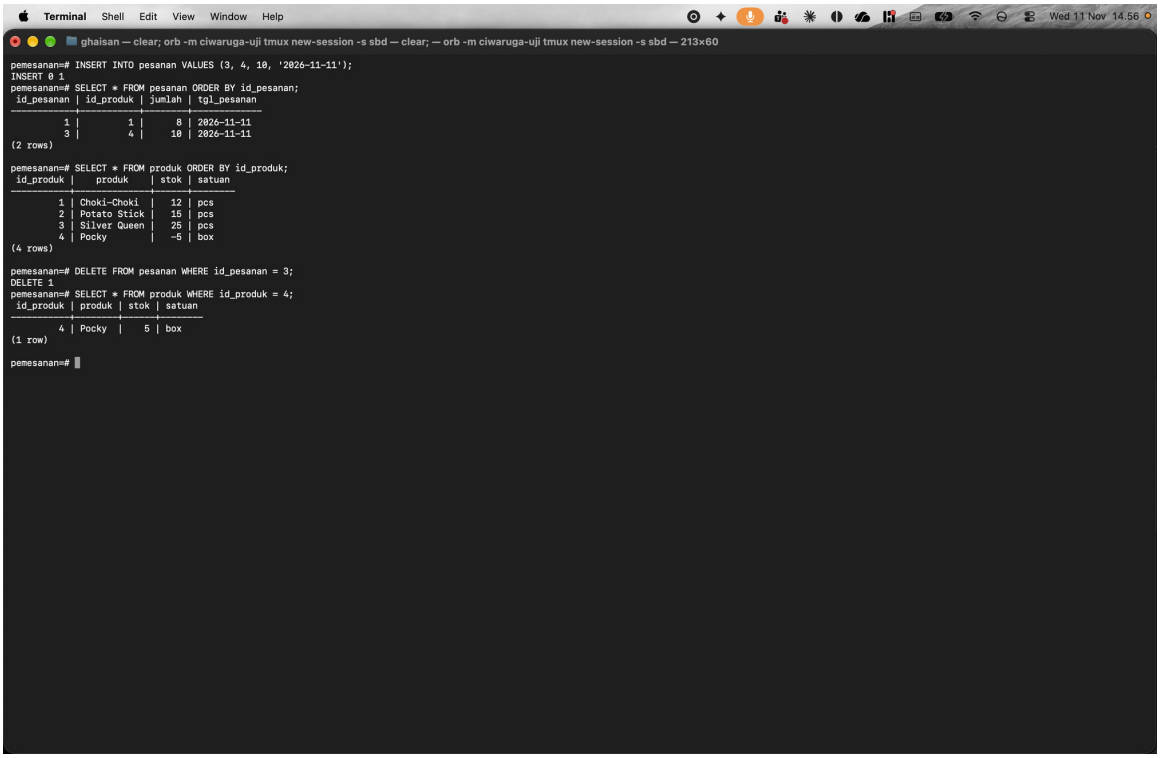
Soal 1: pengujian operasi UPDATE



Soal 1: pengujian operasi DELETE



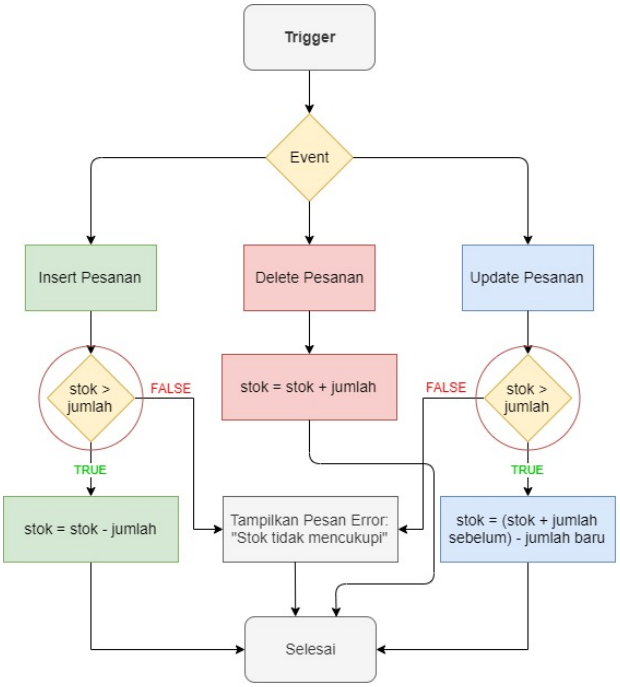
Soal 1 (masalah yang muncul): stok bisa bernilai minus



Soal No. 2

Soal

Business Process #2



Pada Business Process #1, masih terdapat masalah dalam hal pemeriksaan ketersediaan stok yang dapat bernilai kurang dari nol (minus). Maka, diperlukan solusi untuk menjaga ketersediaan stok! Sehingga, diperlukan Business Process #2 untuk menyelesaikan masalah tersebut.

a. Buatlah solusi untuk semua operasi dalam menyelesaikan masalah di atas menggunakan CONSTRAINT CHECK!
b. Lakukan pengujian untuk semua operasi atas penerapan solusi menggunakan CONSTRAINT CHECK!
c. Buatlah solusi untuk semua operasi dalam menyelesaikan masalah di atas menggunakan conditional (IF-ELSE) dan menampilkan pesan error ketika stok tidak tersedia!
d. Lakukan pengujian untuk semua operasi atas penerapan solusi menggunakan conditional (IF-ELSE) dan menampilkan pesan error ketika stok tidak tersedia!

Jawaban (Penjelasan dan Syntax)

Soal 2a: solusi dengan CONSTRAINT CHECK pada kolom stok

Solusi pertama: CONSTRAINT CHECK (stok >= 0) pada tabel produk, sedangkan function trigger tidak diubah sama sekali. Setiap kali trigger menjalankan UPDATE produk, PostgreSQL memeriksa aturan CHECK pada baris hasil UPDATE. Kalau stok jadi minus, UPDATE ditolak.

Trigger berjalan di dalam statement yang sama dengan INSERT/UPDATE pada pesanan, jadi error dari trigger membatalkan seluruh statement, termasuk perubahan di tabel pesanan.

\d produk menunjukkan constraint produk_stok_ck sudah terpasang. Data yang sudah ada (stok 12, 15, 25, 5) lolos pemeriksaan sehingga constraint bisa ditambahkan.

```
ALTER TABLE produk ADD CONSTRAINT produk_stok_ck CHECK (stok >= 0);

\d produk
```

Soal 2b: pengujian CONSTRAINT CHECK untuk operasi INSERT

INSERT pesanan Pocky 10 box (stok 5) ditolak dengan ERROR new row for relation "produk" violates check constraint "produk_stok_ck". DETAIL menunjukkan baris yang ditolak (4, Pocky, -5, box), dan CONTEXT menunjukkan error terjadi di UPDATE di dalam fn_update_stok().
Setelah error, pesanan 3 tidak tersimpan dan stok Pocky tetap 5. INSERT pesanan Pocky 5 box (sama dengan stok) diterima dan stok menjadi 0, karena aturan CHECK hanya melarang stok minus.

```
INSERT INTO pesanan VALUES (3, 4, 10, '2026-11-11');

SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;

INSERT INTO pesanan VALUES (3, 4, 5, '2026-11-11');

SELECT * FROM produk WHERE id_produk = 4;
```

Soal 2b: pengujian CONSTRAINT CHECK untuk operasi UPDATE
Pesanan 1 (Choki-Choki 8 pcs, stok 12) diubah menjadi 30. Stok yang tersedia untuk pesanan ini 12 + 8 = 20, sehingga hasilnya 20 - 30 = -10 dan UPDATE ditolak oleh constraint. Jumlah pesanan 1 tetap 8 dan stok tetap 12.
UPDATE menjadi 10 diterima karena (12 + 8) - 10 = 10.

```
UPDATE pesanan SET jumlah = 30 WHERE id_pesanan = 1;

SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;

UPDATE pesanan SET jumlah = 10 WHERE id_pesanan = 1;

SELECT * FROM produk WHERE id_produk = 1;
```

Soal 2b: pengujian CONSTRAINT CHECK untuk operasi DELETE
DELETE selalu menambah stok, jadi operasi ini tidak mungkin melanggar CHECK. Pesanan 3 (Pocky 5 box) dihapus dan stok Pocky kembali dari 0 menjadi 5.

```
DELETE FROM pesanan WHERE id_pesanan = 3;

SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;
```

Soal 2c: solusi dengan conditional IF-ELSE dan pesan error di function trigger
Solusi kedua: pemeriksaan stok di dalam function trigger memakai IF-ELSE. Constraint produk_stok_ck dihapus dulu supaya pengujian benar-benar membuktikan solusi IF-ELSE, bukan constraint. Trigger trg_update_stok tidak perlu dibuat ulang karena trigger memanggil function berdasarkan nama, jadi isi baru dari CREATE OR REPLACE FUNCTION langsung dipakai.
INSERT: stok produk diambil dulu ke variabel stok_tersedia. Kalau stok_tersedia >= NEW.jumlah stok dikurangi, kalau tidak RAISE EXCEPTION 'Stok tidak mencukupi' beserta angka stok dan jumlah pesanan.
UPDATE: stok yang bisa dipakai adalah stok sekarang ditambah jumlah pesanan lama (OLD.jumlah), karena jumlah lama itu dikembalikan dulu. Nilai itu yang dibandingkan dengan NEW.jumlah.
DELETE: tidak diperiksa karena hanya menambah stok, sama seperti flowchart Business Process #2.
Di flowchart syaratnya ditulis stok > jumlah. Di sini dipakai >= supaya sama dengan aturan CHECK sebelumnya: pesanan sebanyak sisa stok tetap boleh (stok menjadi 0), yang dilarang hanya stok minus.
RAISE EXCEPTION membatalkan statement yang memicu trigger, jadi baris pesanan juga batal tersimpan.

```
ALTER TABLE produk DROP CONSTRAINT produk_stok_ck;
```

```
CREATE OR REPLACE FUNCTION fn_update_stok()
  RETURNS TRIGGER
  LANGUAGE PLPGSQL
  AS
$$
DECLARE
  stok_tersedia NUMERIC;
BEGIN
IF TG_OP = 'INSERT' THEN
  SELECT stok INTO stok_tersedia FROM produk WHERE id_produk = NEW.id_produk;
  IF stok_tersedia >= NEW.jumlah THEN
    UPDATE produk SET stok = (stok - NEW.jumlah) WHERE id_produk = NEW.id_produk;
  ELSE
    RAISE EXCEPTION 'Stok tidak mencukupi (stok tersedia %, jumlah dipesan %)',
      stok_tersedia, NEW.jumlah;
  END IF;
ELSIF TG_OP = 'UPDATE' THEN
  -- stok yang bisa dipakai = stok sekarang + jumlah pesanan sebelum diubah
  SELECT stok + OLD.jumlah INTO stok_tersedia FROM produk WHERE id_produk =
    OLD.id_produk;
  IF stok_tersedia >= NEW.jumlah THEN
    UPDATE produk SET stok = ((stok + OLD.jumlah) - NEW.jumlah) WHERE id_produk =
    OLD.id_produk;
  ELSE
    RAISE EXCEPTION 'Stok tidak mencukupi (stok tersedia %, jumlah dipesan %)',
      stok_tersedia, NEW.jumlah;
  END IF;
ELSIF TG_OP = 'DELETE' THEN
  UPDATE produk SET stok = (stok + OLD.jumlah) WHERE id_produk = OLD.id_produk;
END IF;

RETURN NULL;
END;
$$;
```

Soal 2d: pengujian IF-ELSE untuk operasi INSERT

INSERT Pocky 10 box ditolak dengan pesan ERROR: Stok tidak mencukupi (stok tersedia 5, jumlah dipesan 10). CONTEXT menunjukkan pesan berasal dari RAISE di fn_update_stok(). Pesanan tidak tersimpan dan stok tetap 5.

INSERT Pocky 5 box diterima dan stok menjadi 0.

```
INSERT INTO pesanan VALUES (3, 4, 10, '2026-11-11');

SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;

INSERT INTO pesanan VALUES (3, 4, 5, '2026-11-11');

SELECT * FROM produk WHERE id_produk = 4;
```

Soal 2d: pengujian IF-ELSE untuk operasi UPDATE

Pesanan 1 (Choki-Choki 10 pcs, stok 10) diubah menjadi 25. Stok tersedia 10 + 10 = 20, lebih kecil dari 25, jadi ditolak dengan pesan Stok tidak mencukupi (stok tersedia 20, jumlah dipesan 25) dan data tidak berubah.

UPDATE menjadi 20 diterima karena 20 >= 20, dan stok Choki-Choki menjadi (10 + 10) - 20 = 0.

```
UPDATE pesanan SET jumlah = 25 WHERE id_pesanan = 1;

SELECT * FROM pesanan ORDER BY id_pesanan;

SELECT * FROM produk ORDER BY id_produk;

UPDATE pesanan SET jumlah = 20 WHERE id_pesanan = 1;

SELECT * FROM produk WHERE id_produk = 1;
```

Soal 2d: pengujian IF-ELSE untuk operasi DELETE

Pesanan 3 (Pocky 5 box) dihapus dan stok Pocky kembali dari 0 menjadi 5. DELETE tetap berjalan normal karena tidak melewati pemeriksaan stok.

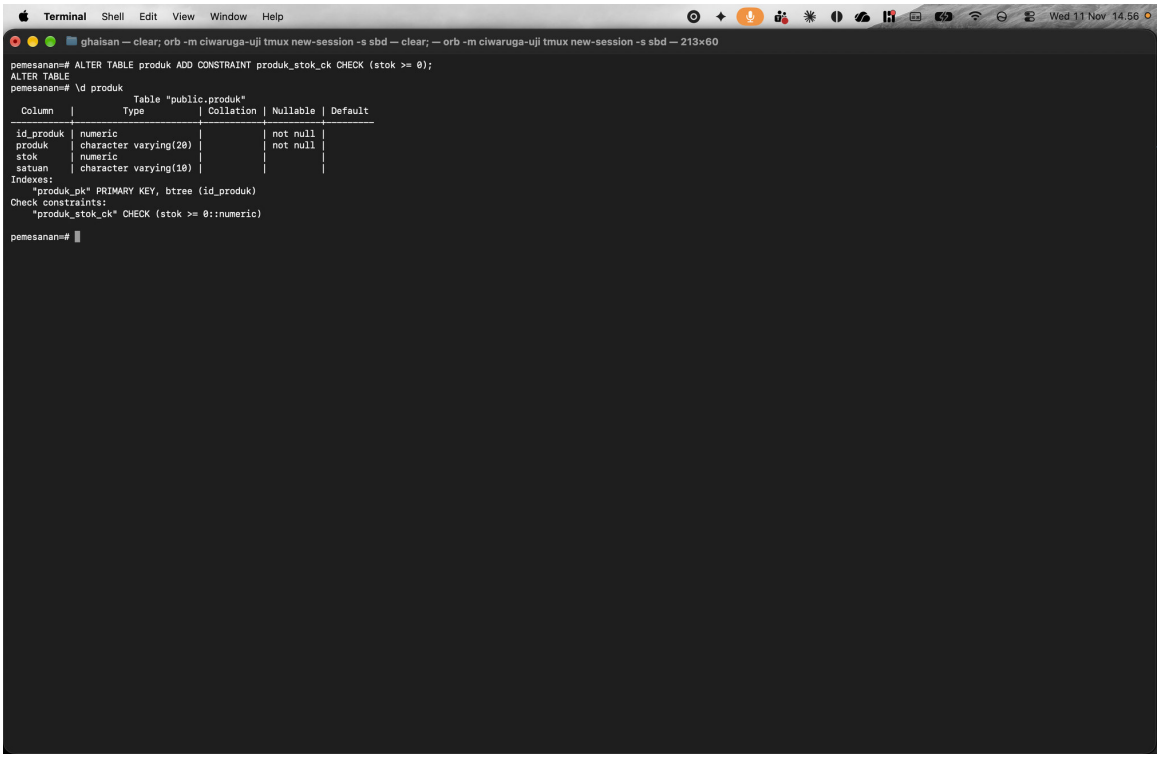
```
DELETE FROM pesanan WHERE id_pesanan = 3;

SELECT * FROM pesanan ORDER BY id_pesanan;

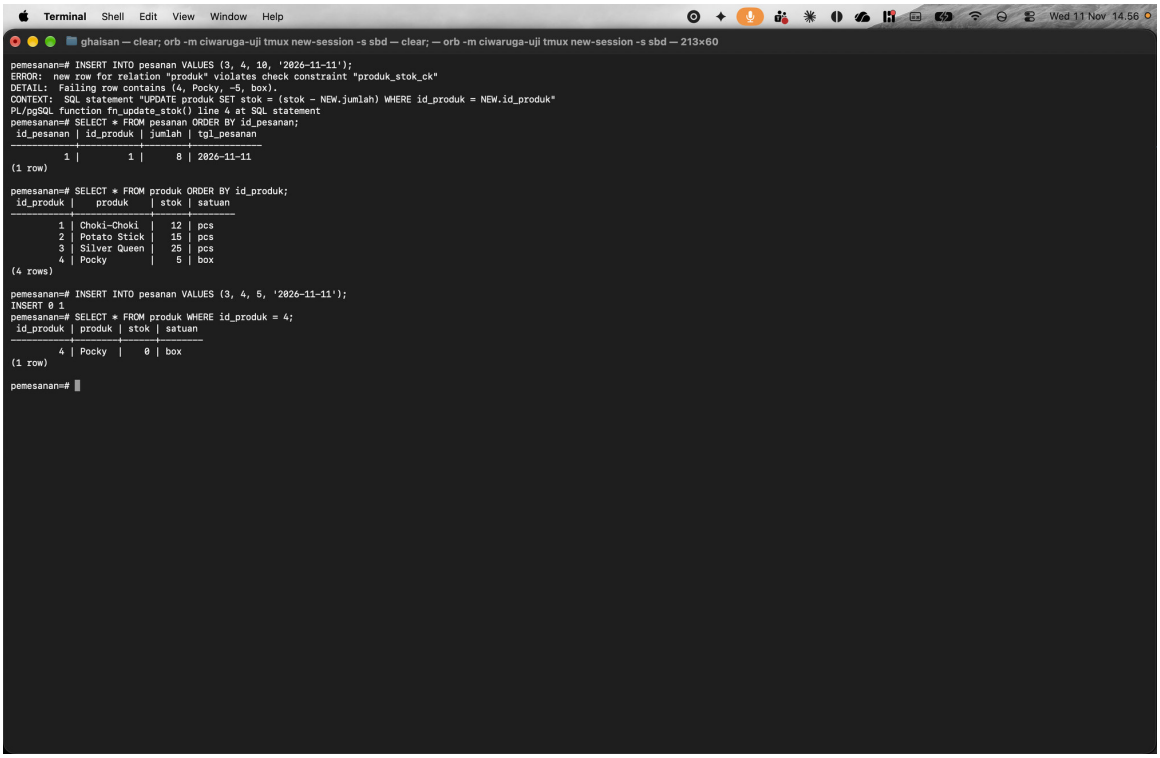
SELECT * FROM produk ORDER BY id_produk;
```

Hasil Eksekusi (Screenshot)

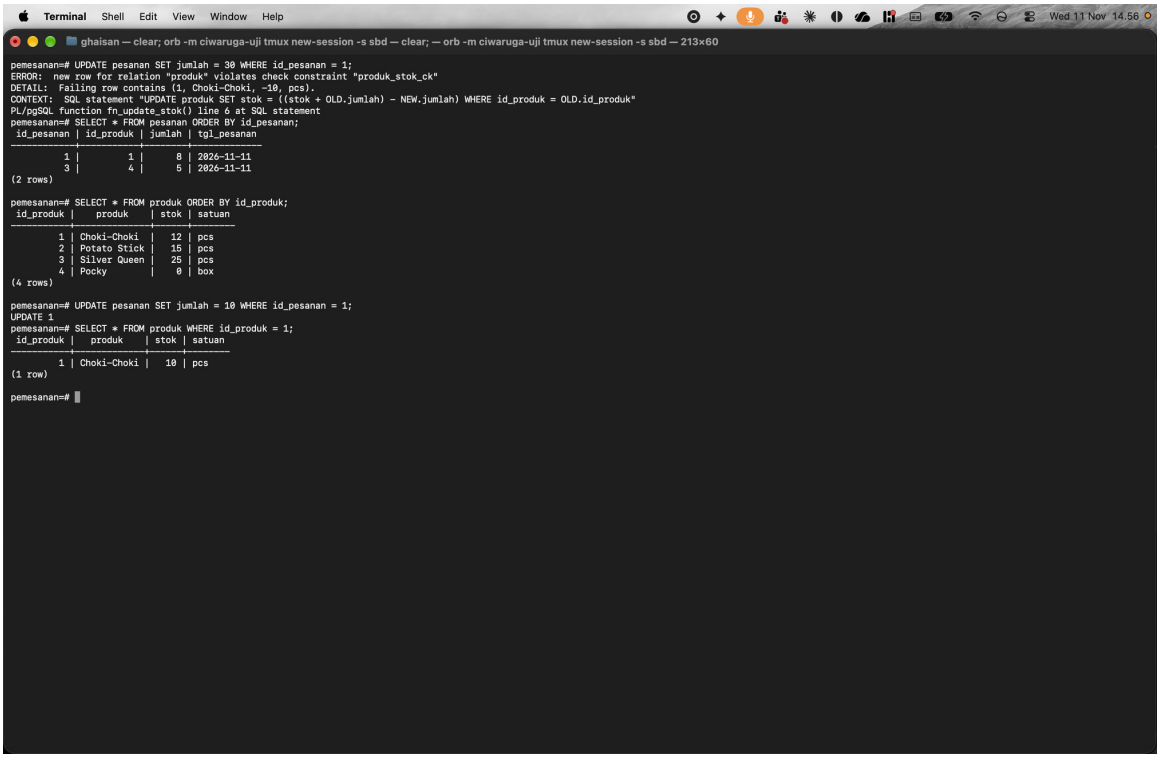
Soal 2a: solusi dengan CONSTRAINT CHECK pada kolom stok



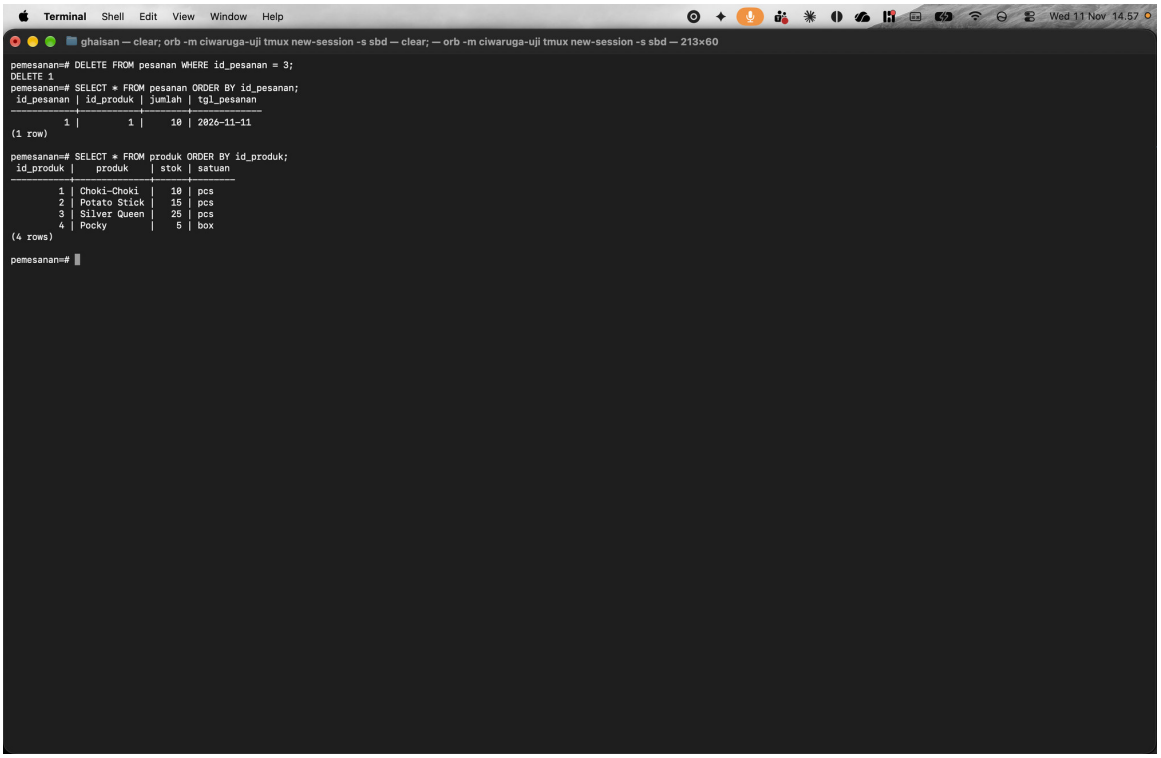
Soal 2b: pengujian CONSTRAINT CHECK untuk operasi INSERT



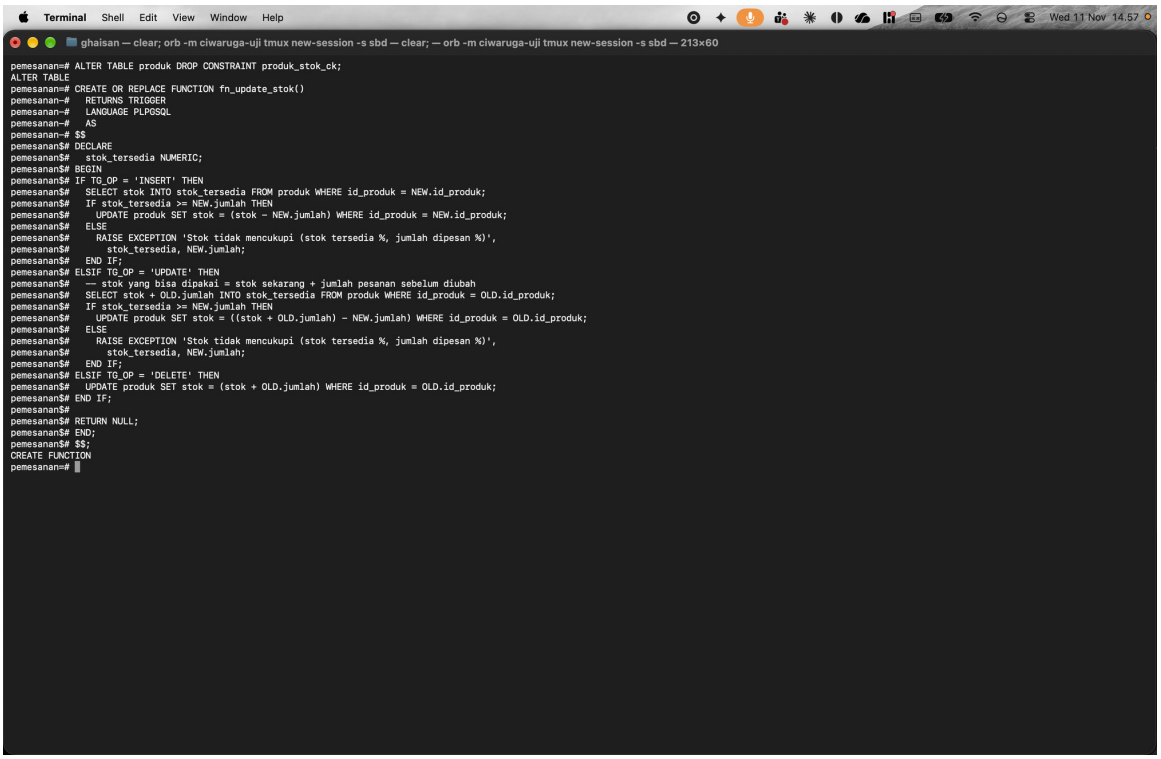
Soal 2b: pengujian CONSTRAINT CHECK untuk operasi UPDATE



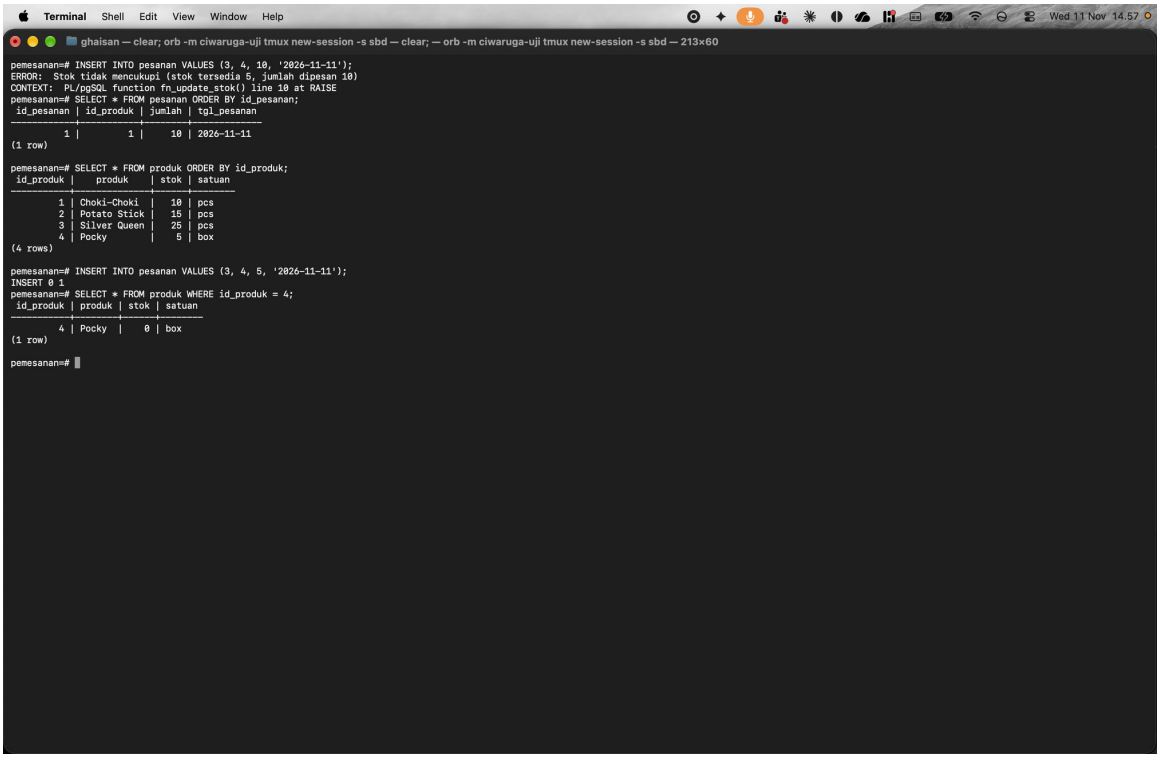
Soal 2b: pengujian CONSTRAINT CHECK untuk operasi DELETE



Soal 2c: solusi dengan conditional IF-ELSE dan pesan error di function trigger



Soal 2d: pengujian IF-ELSE untuk operasi INSERT



```
Terminal  Shell  Edit  View  Window  Help                                  Wed 11 Nov 14.57
ghaisan — clear; orb -m ciwaruga-uji tmux new-session -s sbd — clear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

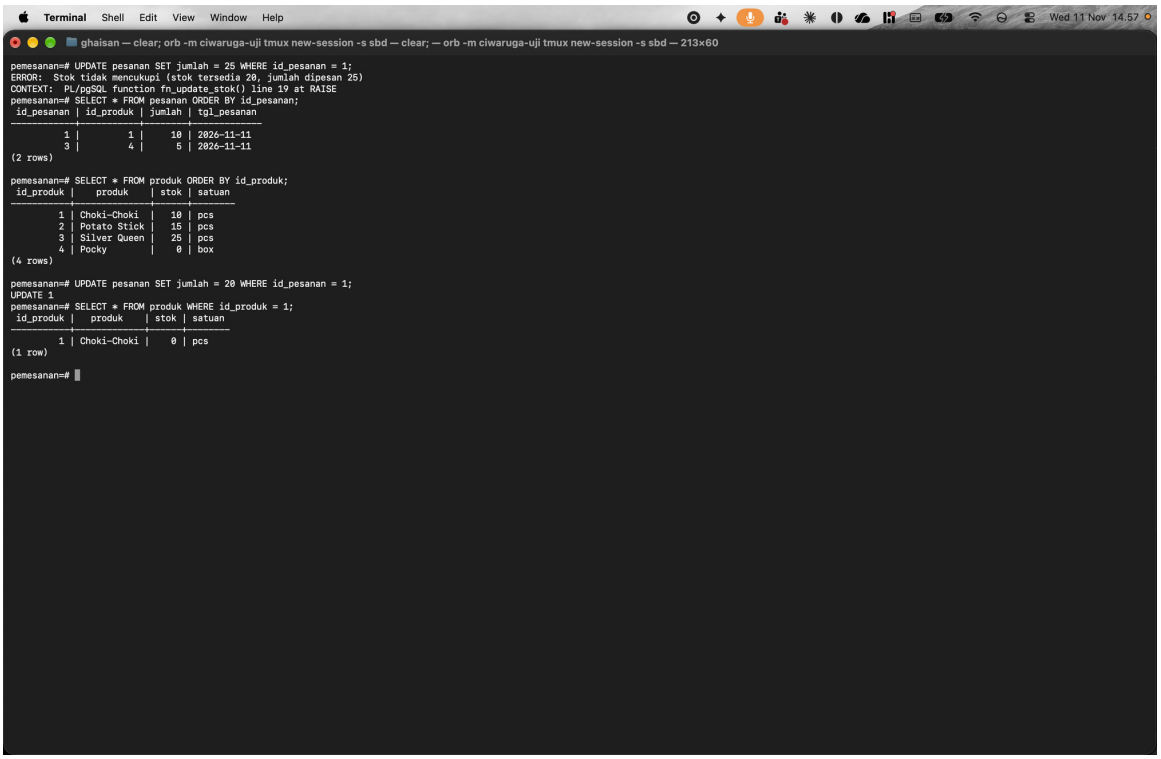
pemesanan=# INSERT INTO pesanan VALUES (3, 4, 10, '2026-11-11');
ERROR:  Stok tidak mencukupi (stok tersedia 5, jumlah dipesan 10)
CONTEXT:  PL/pgSQL function fn_update_stok() line 10 at RAISE
pemesanan=# SELECT * FROM pesanan ORDER BY id_pesanan;
 id_pesanan | id_produk | jumlah | tgl_pesanan
------------+-----------+--------+-------------
          1 |         1 |     10 | 2026-11-11
(1 row)

pemesanan=# SELECT * FROM produk ORDER BY id_produk;
 id_produk |    produk    | stok | satuan
-----------+--------------+------+--------
         1 | Choki-Choki  |   10 | pcs
         2 | Potato Stick |   15 | pcs
         3 | Silver Queen |   25 | pcs
         4 | Pocky        |    5 | box
(4 rows)

pemesanan=# INSERT INTO pesanan VALUES (3, 4, 5, '2026-11-11');
INSERT 0 1
pemesanan=# SELECT * FROM produk WHERE id_produk = 4;
 id_produk | produk | stok | satuan
-----------+--------+------+--------
         4 | Pocky  |    0 | box
(1 row)

pemesanan=#
```

Soal 2d: pengujian IF-ELSE untuk operasi UPDATE



```
Terminal  Shell  Edit  View  Window  Help                                  Wed 11 Nov 14.57
ghaisan — clear; orb -m ciwaruga-uji tmux new-session -s sbd — clear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

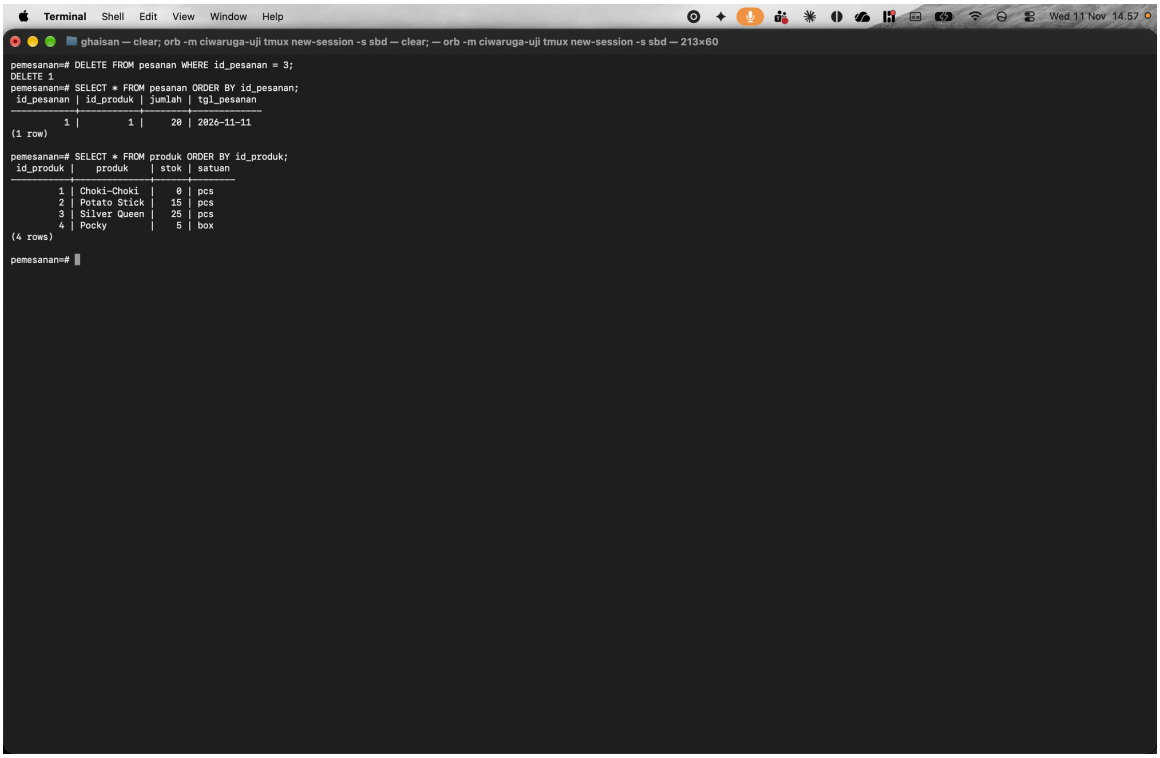
pemesanan=# UPDATE pesanan SET jumlah = 25 WHERE id_pesanan = 1;
ERROR:  Stok tidak mencukupi (stok tersedia 20, jumlah dipesan 25)
CONTEXT:  PL/pgSQL function fn_update_stok() line 19 at RAISE
pemesanan=# SELECT * FROM pesanan ORDER BY id_pesanan;
 id_pesanan | id_produk | jumlah | tgl_pesanan
------------+-----------+--------+-------------
          1 |         1 |     10 | 2026-11-11
          3 |         4 |      5 | 2026-11-11
(2 rows)

pemesanan=# SELECT * FROM produk ORDER BY id_produk;
 id_produk |    produk    | stok | satuan
-----------+--------------+------+--------
         1 | Choki-Choki  |   10 | pcs
         2 | Potato Stick |   15 | pcs
         3 | Silver Queen |   25 | pcs
         4 | Pocky        |    0 | box
(4 rows)

pemesanan=# UPDATE pesanan SET jumlah = 20 WHERE id_pesanan = 1;
UPDATE 1
pemesanan=# SELECT * FROM produk WHERE id_produk = 1;
 id_produk |   produk    | stok | satuan
-----------+-------------+------+--------
         1 | Choki-Choki |    0 | pcs
(1 row)

pemesanan=#
```

Soal 2d: pengujian IF-ELSE untuk operasi DELETE



Soal No. 3

Soal
a. Apakah Trigger dapat memanggil Function? Sertakan referensinya! b. Apakah Trigger dapat memanggil Procedure? Lakukan percobaan untuk mengujinya langsung, dan sertakan referensinya!
Jawaban (Penjelasan dan Syntax)

Soal 3a: trigger memanggil function

Jawaban: ya, trigger memanggil function. Di PostgreSQL trigger tidak berisi kode sendiri, yang dijalankan adalah function yang disebut di bagian EXECUTE FUNCTION. Function itu harus tanpa parameter dan RETURNS TRIGGER (dokumentasi CREATE TRIGGER, bagian function_name).

Query ke katalog pg_trigger menunjukkan trigger trg_update_stok pada tabel pesanan terhubung ke function fn_update_stok.

Percobaan tambahan: function biasa fn_jumlah_pesanan yang RETURNS INTEGER ditolak saat dipasang ke trigger dengan pesan function fn_jumlah_pesanan must return type trigger. Jadi function yang bisa dipanggil trigger hanya function trigger.

Referensi: https://www.postgresql.org/docs/current/sql-createtrigger.html dan https://www.postgresql.org/docs/current/plpgsql-trigger.html

```
SELECT tgname AS trigger, tgrelid::regclass AS tabel, tgfoid::regproc AS function
FROM pg_trigger
WHERE NOT tgisinternal;

CREATE OR REPLACE FUNCTION fn_jumlah_pesanan()
  RETURNS INTEGER
  LANGUAGE plpgsql
AS $$
BEGIN
  RETURN (SELECT COUNT(*) FROM pesanan);
END;
```

```
$$;

CREATE TRIGGER trg_coba_function
  AFTER INSERT ON pesanan
  FOR EACH ROW
  EXECUTE FUNCTION fn_jumlah_pesanan();
```

Soal 3b (percobaan 1): trigger memanggil procedure secara langsung

Jawaban: tidak bisa secara langsung. Procedure pr_coba berhasil dibuat, tetapi waktu dipasang ke trigger dengan EXECUTE PROCEDURE pr_coba() keluar ERROR: function pr_coba() does not exist. Hasilnya sama dengan EXECUTE FUNCTION pr_coba().

Kata PROCEDURE di EXECUTE PROCEDURE hanya sisa sintaks lama. Menurut dokumentasi CREATE TRIGGER, kata FUNCTION dan PROCEDURE di bagian itu artinya sama, dan yang dirujuk tetap harus function, bukan procedure. PostgreSQL hanya mencari function bernama pr_coba, sehingga procedure tidak ditemukan.

```
CREATE OR REPLACE PROCEDURE pr_coba()
  LANGUAGE plpgsql
AS $$
BEGIN
  RAISE NOTICE 'procedure pr_coba dipanggil';
END;
$$;

CREATE TRIGGER trg_coba_procedure
  AFTER INSERT ON pesanan
  FOR EACH ROW
  EXECUTE PROCEDURE pr_coba();

CREATE TRIGGER trg_coba_procedure
  AFTER INSERT ON pesanan
  FOR EACH ROW
  EXECUTE FUNCTION pr_coba();
```

Soal 3b (percobaan 2): procedure dipanggil dengan CALL dari dalam function trigger

Walaupun tidak bisa langsung, procedure tetap bisa dijalankan oleh trigger lewat perantara function trigger. Dibuat tabel riwayat_pesanan, procedure pr_catat_riwayat yang menyimpan catatan ke tabel itu, function trigger fn_catat_riwayat yang memanggil procedure dengan CALL, dan trigger trg_catat_riwayat pada tabel pesanan.

Untuk DELETE yang dipakai OLD.id_pesanan karena NEW kosong, sedangkan untuk INSERT dan UPDATE dipakai NEW.id_pesanan.

```
CREATE TABLE riwayat_pesanan (
    id_riwayat SERIAL PRIMARY KEY,
    id_pesanan NUMERIC,
    aksi VARCHAR(10),
    waktu TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE OR REPLACE PROCEDURE pr_catat_riwayat(p_id_pesanan NUMERIC, p_aksi VARCHAR)
  LANGUAGE plpgsql
AS $$
BEGIN
  INSERT INTO riwayat_pesanan (id_pesanan, aksi) VALUES (p_id_pesanan, p_aksi);
  RAISE NOTICE 'procedure pr_catat_riwayat dipanggil: pesanan %, aksi %',
    p_id_pesanan, p_aksi;
END;
$$;

CREATE OR REPLACE FUNCTION fn_catat_riwayat()
  RETURNS TRIGGER
  LANGUAGE plpgsql
AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    CALL pr_catat_riwayat(OLD.id_pesanan, TG_OP);
  ELSE
```

```
    CALL pr_catat_riwayat(NEW.id_pesanan, TG_OP);
  END IF;
  RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_catat_riwayat
  AFTER INSERT OR UPDATE OR DELETE ON pesanan
  FOR EACH ROW
  EXECUTE FUNCTION fn_catat_riwayat();
```

Soal 3b (percobaan 2): pengujian trigger yang memanggil procedure lewat function

INSERT, UPDATE, dan DELETE pesanan 4 masing-masing memunculkan NOTICE dari dalam procedure, dan tabel riwayat_pesanan berisi 3 catatan (INSERT, UPDATE, DELETE). Trigger trg_update_stok tetap jalan bersamaan: stok Potato Stick sempat berkurang lalu kembali ke 15 setelah pesanan dihapus.

Kesimpulan: trigger hanya bisa memanggil function (RETURNS TRIGGER) secara langsung, sedangkan procedure hanya bisa dipanggil dari dalam function trigger memakai CALL. Procedure yang dipanggil dari trigger juga tidak boleh menjalankan COMMIT atau ROLLBACK, karena trigger selalu berada di dalam transaksi statement pemicunya.

Referensi: https://www.postgresql.org/docs/current/sql-createtrigger.html dan https://www.postgresql.org/docs/current/sql-call.html

```
INSERT INTO pesanan VALUES (4, 2, 3, '2026-11-11');

UPDATE pesanan SET jumlah = 4 WHERE id_pesanan = 4;

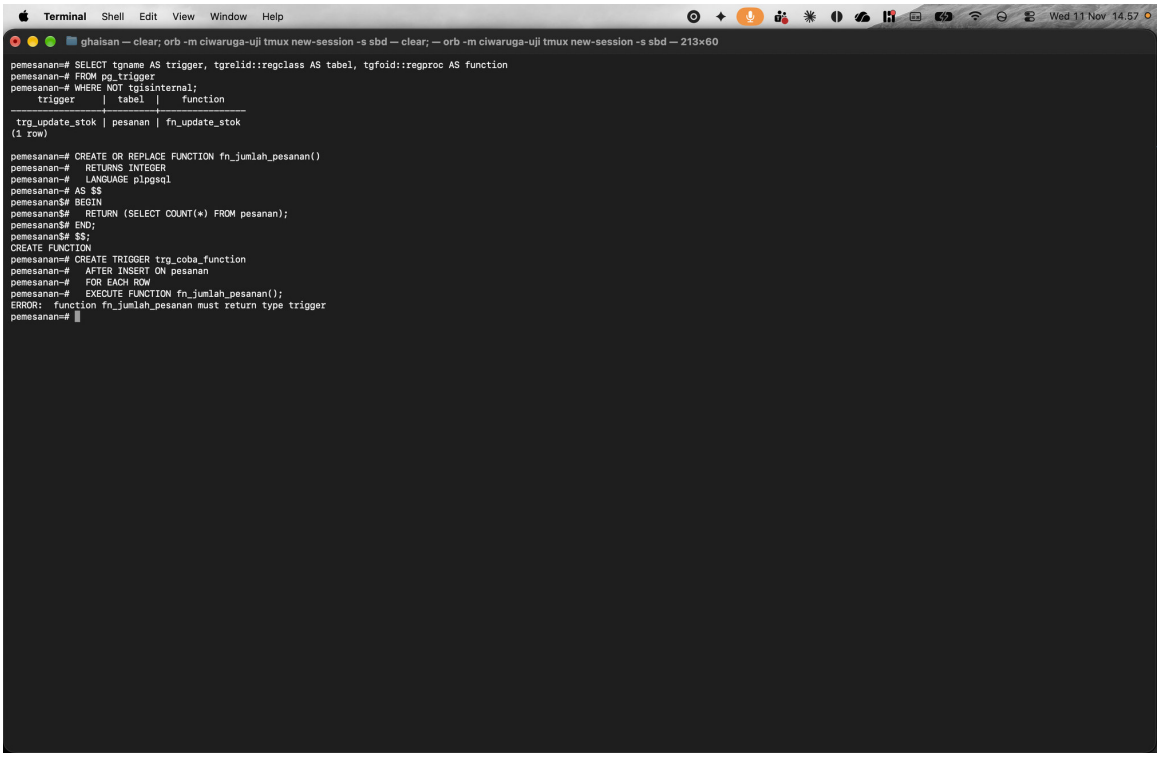
DELETE FROM pesanan WHERE id_pesanan = 4;

SELECT id_riwayat, id_pesanan, aksi FROM riwayat_pesanan ORDER BY id_riwayat;

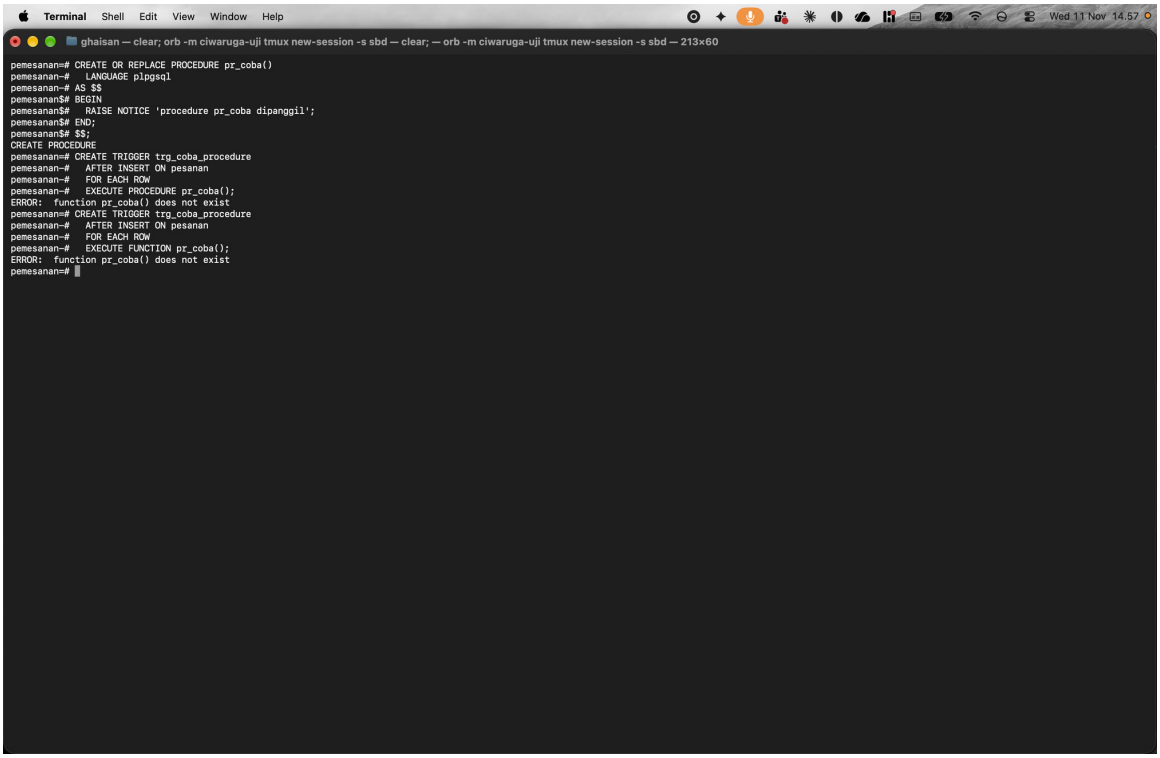
SELECT * FROM produk WHERE id_produk = 2;
```

Hasil Eksekusi (Screenshot)

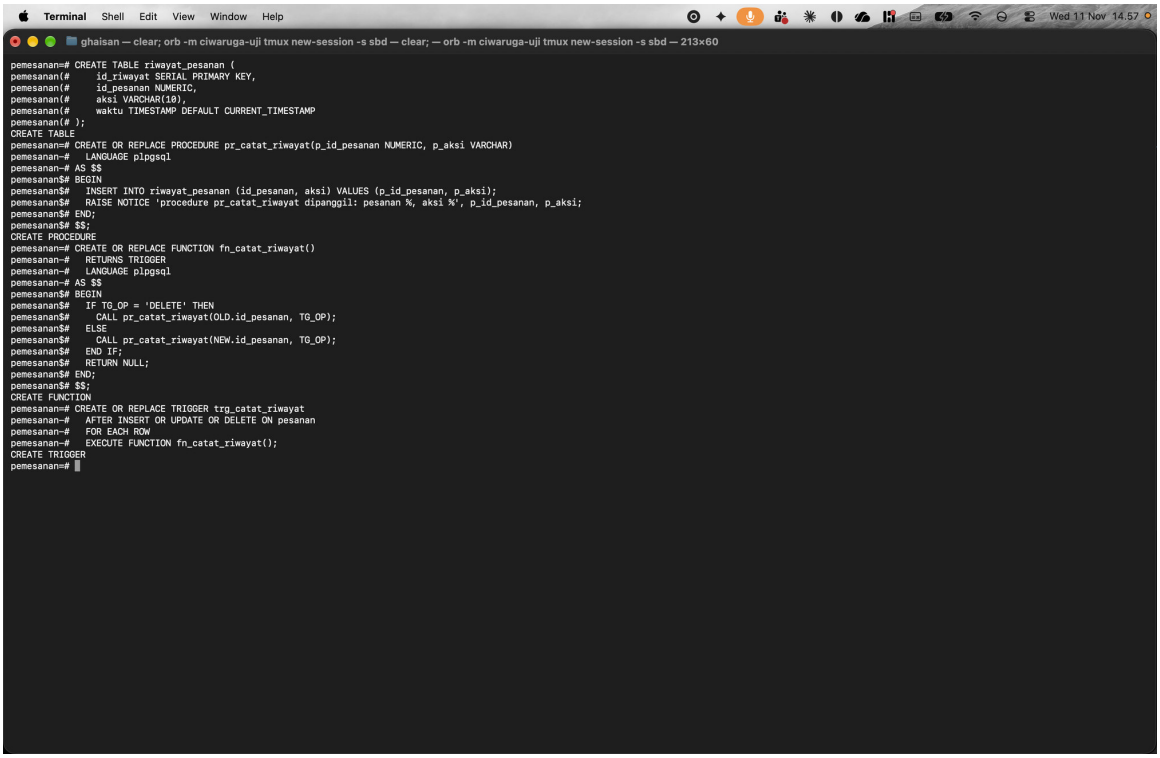
Soal 3a: trigger memanggil function



Soal 3b (percobaan 1): trigger memanggil procedure secara langsung



Soal 3b (percobaan 2): procedure dipanggil dengan CALL dari dalam function trigger



Soal 3b (percobaan 2): pengujian trigger yang memanggil procedure lewat function

